LA(1)LR(0)

임기정

Abstract

This note fixes the LR(0) characteristic automaton, defines the one-symbol lookahead set attached to each completed LR(0) item, and derives the usual Read/Follow digraph computation. The proofs are written so that the essential claims appear as inclusions, fixed-point equations, and derivation chains.

Contents

1 The LR(0) Automaton

Definition 1 (LRA(G))**.** Let $G \triangleq (N, T, P, S)$ be a context-free grammar. Define:

(i)
$$G' = (N', T', P', S') \triangleq (N \uplus \{S'\},\ T \uplus \{\$\},\ P \cup \{[S' \to S\$]\},\ S').$$

(ii)
$$V \triangleq N \uplus T, \qquad V' \triangleq N' \uplus T'.$$

(iii)
$$\beta \underset{\mathbf{rm}}{\overset{}{\Longrightarrow}}_{P'} \gamma \quad \overset{\text{def}}{\iff} \quad (\beta, \gamma) \in \{(\alpha A z, \alpha\omega z) \mid [A \to \omega] \in P',\ \alpha \in V'^*,\ z \in T'^*\}.$$

(iv)
$$I_{\mathrm{LR}(0)} \triangleq \{[A \to \alpha \cdot \beta] \mid [A \to \alpha\beta] \in P'\}.$$

(v) For $q \subseteq I_{\mathrm{LR}(0)}$,
$$\mathrm{Cl}(q) =_\mu q \cup \{[A \to \varepsilon \cdot \omega] \mid [A \to \omega] \in P',\ [B \to \beta \cdot A\gamma] \in \mathrm{Cl}(q)\}.$$

(vi)
$$q_0 \triangleq \mathrm{Cl}(\{[S' \to \varepsilon \cdot S\$]\}).$$

(vii) For $q \subseteq I_{\mathrm{LR}(0)}$ and $X \in V'$,
$$\mathrm{GOTO}(q, X) \triangleq \mathrm{Cl}(\{[A \to \alpha X \cdot \beta] \mid [A \to \alpha \cdot X\beta] \in q\}).$$

(viii)
$$Q \triangleq \mathrm{PT} \setminus \{\emptyset\}, \qquad \mathrm{PT} =_\mu \{q_0\} \cup \{\mathrm{GOTO}(q, X) \mid q \in \mathrm{PT},\ X \in V'\}.$$

(ix) The path relation is defined by structural recursion on the input word:

$$\varepsilon : p \to q \iff p \in Q \wedge p = q, \qquad X\alpha : p \to q \iff p \in Q \wedge \mathrm{GOTO}(p, X) \in Q \wedge \alpha : \mathrm{GOTO}(p, X) \to q.$$

Thus a path blocks exactly when the next GOTO-state is empty; the relation is deterministic, not an additional least fixed point.

(x)
$$\mathbf{Config} \triangleq \{(\alpha : p \to q, z) \mid \alpha \in V'^*,\ p, q \in Q,\ z \in T'^*,\ \alpha : p \to q\}.$$

(xi)
$$\delta(q, X) \triangleq \mathrm{GOTO}(q, X) \qquad (q \in Q,\ X \in V').$$

(xii)
$$\mathbf{reduce}(q, t) \triangleq \{[A \to \omega] \mid [A \to \omega \cdot \varepsilon] \in q\} \qquad (q \in Q,\ t \in T').$$

(xiii) Let $\vdash \subseteq \mathbf{Config} \times \mathbf{Config}$ be generated by:

$$\frac{q'' = \delta(q', t) \quad q'' \in Q}{(\alpha : q \to q', tz) \vdash (\alpha t : q \to q'', z)}\ \text{Shift}$$

$$\frac{\alpha : q \to p \quad \omega : p \to q' \quad [A \to \omega] \in \mathbf{reduce}(q', t) \quad q'' = \delta(p, A) \quad q'' \in Q}{(\alpha\omega : q \to q', tz) \vdash (\alpha A : q \to q'', tz)}\ \text{Reduce}(A \to \omega)$$

(xiv)
$$q_f \triangleq \delta(\delta(q_0, S), \$).$$

(xv) The accepted language is defined as

$$L(\mathrm{LRA}(G)) \triangleq \{z \in T^* \mid (\varepsilon : q_0 \to q_0, z\$) \vdash^* (S\$: q_0 \to q_f, \varepsilon)\}.$$

For $c = (\alpha : p \to q, u) \in \mathbf{Config}$, define the unread input by

$$\mathrm{input}(c) \triangleq u.$$

The accepting configurations are exactly

$$\mathsf{Accept}(c) \iff c = (S\$: q_0 \to q_f, \varepsilon),$$

and an error configuration is a non-accepting run configuration with no Shift or Reduce successor under the table being executed. In a mechanized definition, the displayed stack path is proof-carrying: the constructors for Shift and Reduce above include the side conditions that their target states lie in Q, so their conclusions are again elements of **Config**.

For $c = (\alpha : p \to q, u) \in \mathbf{Config}$, set $\mathrm{Y}(c) \in V'^*$ as

$$\mathrm{Y}(c) \triangleq \alpha u.$$

Lemma 2 (One-step yield invariant)**.** *Each parser step reverses zero or one rightmost grammar step:*

$$c \vdash c' \Longrightarrow \begin{cases} \mathrm{Y}(c') = \mathrm{Y}(c), & \text{if the step is Shift,} \\ \mathrm{Y}(c') \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \mathrm{Y}(c), & \text{if the step is Reduce.} \end{cases} \tag{1}$$

Proof. Indeed,

$$\begin{aligned} \text{Shift}: \quad \mathrm{Y}(\alpha : q \to q', tz) &= \alpha tz &&= \mathrm{Y}(\alpha t : q \to q'', z), \\ \text{Reduce}: \quad \mathrm{Y}(\alpha A : q \to q'', tz) &= \alpha Atz &&\underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha\omega tz = \mathrm{Y}(\alpha\omega : q \to q', tz). \end{aligned}$$

□

Corollary 3 (Multi-step yield invariant)**.** *For $c, c' \in \mathbf{Config}$,*

$$c \vdash^* c' \Longrightarrow \mathrm{Y}(c') \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \mathrm{Y}(c). \tag{2}$$

Proof. Induct on the length of $c \vdash^* c'$ and use Lemma 2 at each parser step. □

Corollary 4 (LR(0) soundness)**.**

$$L(\mathrm{LRA}(G)) \subseteq L(G).$$

Proof.

$$\begin{aligned} z \in L(\mathrm{LRA}(G)) &\Longrightarrow (\varepsilon : q_0 \to q_0, z\$) \vdash^* (S\$: q_0 \to q_f, \varepsilon) \\ &\overset{(2)}{\Longrightarrow} S\$ \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} z\$ \\ &\Longrightarrow S \Rightarrow^*_P z \\ &\Longrightarrow z \in L(G). \end{aligned}$$

The third implication deletes the endmarker. Write the rightmost derivation witnessing $S\$ \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} z\$$ as

$$S\$ = \gamma_0 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_1 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \cdots \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_M = z\$.$$

Since S' occurs on no right-hand side of P', the only production mentioning S', namely $[S' \to S\$]$, is never applicable to a form derived from $S\$$; hence every step uses a production of $P \subseteq P'$. Moreover $\$ \notin V$ occurs on no right-hand side of P, and no production of P rewrites a terminal, so the rightmost symbol $\$$ of γ_0 is never touched:

$$\gamma_k = \delta_k\$, \qquad \delta_k \in V^* \quad (0 \le k \le M), \qquad \delta_0 = S, \quad \delta_M = z, \qquad \delta_k \Rightarrow_P \delta_{k+1}.$$

Stripping the common trailing $\$$ therefore yields $S = \delta_0 \Rightarrow_P \delta_1 \Rightarrow_P \cdots \Rightarrow_P \delta_M = z$, i.e. $S \Rightarrow^*_P z$. □

Fact 5 (Elimination of non-productive nonterminals)**.** *Call $A \in N'$* productive *if $A \Rightarrow_{P'}^{*} w$ for some $w \in T'^{*}$, and* non-productive *otherwise. Removing every non-productive nonterminal together with all productions that mention one preserves the generated language.*

Proof. Extend productivity to strings: $\sigma \in V'^{*}$ is productive if $\sigma \Rightarrow_{P'}^{*} x$ for some $x \in T'^{*}$. A terminal derives only itself and a derivation of a string is the concatenation of derivations of its symbols, so

$$Y_1 \cdots Y_m \text{ productive} \iff \forall i.\ Y_i \text{ productive} \qquad (t \in T' \text{ productive}). \tag{3}$$

Fixed-point characterization. Let $\mathrm{Gen} \subseteq N'$ be the least fixed point

$$\mathrm{Gen} = \bigcup_{n \geq 0} \mathrm{Gen}_n, \quad \mathrm{Gen}_0 = \emptyset, \quad \mathrm{Gen}_{n+1} = \{A \in N' \mid \exists\, [A \to X_1 \cdots X_k] \in P'.\ \forall i.\ X_i \in T' \cup \mathrm{Gen}_n\}$$

(with $k = 0$, i.e. $[A \to \varepsilon] \in P'$, permitted). Then A is productive iff $A \in \mathrm{Gen}$:

$$\begin{aligned} A \in \mathrm{Gen}_{n+1} &\Longrightarrow [A \to X_1 \cdots X_k] \in P' \text{ with } X_i \in T' \cup \mathrm{Gen}_n \\ &\overset{\mathrm{IH}_n}{\Longrightarrow} A \Rightarrow_{P'} X_1 \cdots X_k \Rightarrow_{P'}^{*} x_1 \cdots x_k \in T'^{*}; \\ A \Rightarrow_{P'}^{\ell} w \in T'^{*}\ (\ell \geq 1) &\Longrightarrow A \Rightarrow_{P'} X_1 \cdots X_k \Rightarrow_{P'}^{\ell-1} w,\ X_i \Rightarrow_{P'}^{\ell_i} w_i,\ \ell_i < \ell \\ &\overset{\mathrm{IH}_\ell}{\Longrightarrow} X_i \in T' \cup \mathrm{Gen} \Longrightarrow A \in \mathrm{Gen}. \end{aligned}$$

Thus the non-productive nonterminals are exactly $N' \setminus \mathrm{Gen}$, a least fixed point.

Language preservation. Let $\widehat{G}$ delete $N' \setminus \mathrm{Gen}$ and every production mentioning it; since terminals are productive, its productions are

$$\widehat{P} \triangleq \{[A \to X_1 \cdots X_k] \in P' \mid A, X_1, \ldots, X_k \text{ all productive}\} \subseteq P'.$$

As $\widehat{P} \subseteq P'$, $S' \Rightarrow_{\widehat{P}}^{*} z$ implies $S' \Rightarrow_{P'}^{*} z$. For the converse, induct on the length ℓ of $\sigma \Rightarrow_{P'}^{\ell} x \in T'^{*}$:

$$\sigma \Rightarrow_{P'}^{\ell} x \in T'^{*} \implies \sigma \text{ productive} \ \wedge\ \sigma \Rightarrow_{\widehat{P}}^{*} x.$$

For $\ell = 0$, $\sigma = x \in T'^{*}$. For $\ell \geq 1$, split off the first step $\sigma = \sigma_1 C \sigma_2$, $[C \to \gamma] \in P'$, $\sigma_1, \sigma_2 \in V'^{*}$:

$$\sigma = \sigma_1 C \sigma_2 \Rightarrow_{P'} \sigma_1 \gamma \sigma_2 \Rightarrow_{P'}^{\ell-1} x \ \overset{\mathrm{IH}}{\Longrightarrow}\ \sigma_1 \gamma \sigma_2 \text{ productive} \ \wedge\ \sigma_1 \gamma \sigma_2 \Rightarrow_{\widehat{P}}^{*} x.$$

By (3), γ is productive, hence so is C (via $[C \to \gamma]$), so $[C \to \gamma] \in \widehat{P}$ and

$$\sigma = \sigma_1 C \sigma_2 \Rightarrow_{\widehat{P}} \sigma_1 \gamma \sigma_2 \Rightarrow_{\widehat{P}}^{*} x.$$

Taking $\sigma = S'$ gives $L(G') \subseteq L(\widehat{G})$, so $L(\widehat{G}) = L(G')$ and every nonterminal of $\widehat{G}$ is productive.

When $L(G) \neq \emptyset$, S is productive, hence S' is productive ($[S' \to S\$]$) and $\widehat{G}$ keeps the start symbol with the same language. This is a language-level replacement only. It must not be used as a "without loss of generality" step for statements about the LR(0) automaton itself, because Cl, GOTO, Q, and all lookahead relations are rebuilt from the production set. Any theorem below that needs productivity states that hypothesis explicitly. □

The path relation factors through the state reached after any prefix: induction on $|\alpha|$ using Definition 1(ix) gives, for all $\alpha, \omega \in V'^{*}$ and $r \in Q$,

$$\alpha\omega : r \to q \iff \exists!\, p \in Q.\ \alpha : r \to p \ \wedge\ \omega : p \to q, \tag{4}$$

the intermediate state being the unique state obtained by following α from r, since each $\delta(\cdot, X) = \mathrm{GOTO}(\cdot, X)$ is single-valued.

Lemma 6 (Finite closed LR(0) states)**.** *The LR(0) construction satisfies the following structural invariants.*

(i) $I_{\mathrm{LR}(0)}$ *is finite, every* $q \in Q$ *is a subset of* $I_{\mathrm{LR}(0)}$*, and* Q *is finite.*

(ii) *Every* $q \in Q$ *is* Cl*-closed: if* $[B \to \beta \cdot A\gamma] \in q$ *and* $[A \to \omega] \in P'$*, then* $[A \to \varepsilon \cdot \omega] \in q$*.*

(iii) *If* $q \in Q$*,* $X \in V'$*, and* $\delta(q, X) \neq \emptyset$*, then* $\delta(q, X) \in Q$*. Equivalently, a nonempty target produced during the closure construction is one of the states of* Q*.*

Proof. The item set $I_{\mathrm{LR}(0)}$ is finite because P' is finite and each production has finitely many dot positions. The operator defining Cl only adds LR(0) items, so $\mathrm{Cl}(q) \subseteq I_{\mathrm{LR}(0)}$ whenever $q \subseteq I_{\mathrm{LR}(0)}$. Hence the fixed point PT lives inside the finite lattice $\mathcal{P}(I_{\mathrm{LR}(0)})$, and $Q = \mathrm{PT} \setminus \{\emptyset\}$ is finite. Closure-closedness is just unfolding the least fixed point defining Cl, and (iii) follows from how PT is closed under all GOTO successors. □

Lemma 7 (Derivation surgery used below)**.** *The following context-free derivation facts are used as ordinary lemmas in the subsequent proofs.*

(i) Rightmostization for terminal targets. *If* $\sigma \Rightarrow_{P'}^{*} x \in T'^{*}$*, then* $\sigma \underset{\mathbf{rm}}{\Longrightarrow}_{P'}^{*} x$*.*

(ii) Terminal right-context insertion. *If* $\sigma \underset{\mathbf{rm}}{\Longrightarrow}_{P'}^{*} \tau$ *and* $z \in T'^{*}$*, then* $\lambda\sigma z \underset{\mathbf{rm}}{\Longrightarrow}_{P'}^{*} \lambda\tau z$ *for every left context* $\lambda \in V'^{*}$*.*

(iii) Terminal concatenation decomposition. *If* $\sigma_1\sigma_2 \Rightarrow_{P'}^{*} x \in T'^{*}$*, then* $x = x_1x_2$ *for some* $x_1, x_2 \in T'^{*}$ *with* $\sigma_i \Rightarrow_{P'}^{*} x_i$*.*

(iv) FIRST/path extraction. *Suppose a state* r *contains an item with the dot before a symbol* X*, and* $X \Rightarrow_{P'}^{*} t\chi$ *for* $t \in T'$*. Then, after following the nullable prefix of the leftmost spine witnessing this derivation, the LR(0) automaton has a nonempty* t*-transition. Equivalently, the terminal* t *obtained from* $\mathrm{FIRST}(X)$ *produces the* $\delta(\cdot, t) \neq \emptyset$ *premise used in Lemma 15.*

Proof. Items (i)–(iii) are the standard induction arguments on derivation length, using the fact that terminals are never rewritten. For (iv), take a leftmost spine from X to the first terminal t. The closure rule inserts the item for each nonterminal encountered on this spine, while the nullable siblings before the spine are followed by GOTO-steps. The first terminal item therefore gives a nonempty t-successor. In Coq these four statements are best made explicit once and then reused rather than reproved inside the main lookahead proof. □

Definition 8 (Viable prefixes and valid items)**.** For $\eta \in V'^{*}$, say that η is a *viable prefix* of G' when it ends at some dot position of a handle in a right-sentential form:

$$\eta \text{ viable} \iff \exists A, \alpha, \beta, \delta, w.\ [A \to \alpha\beta] \in P' \wedge \eta = \delta\alpha \wedge w \in T'^{*} \wedge S' \underset{\mathbf{rm}}{\Longrightarrow}_{P'}^{*} \delta A w. \tag{5}$$

For the same η, define the valid LR(0) items by

$$[A \to \alpha \cdot \beta] \in \mathsf{Valid}(\eta) \iff [A \to \alpha\beta] \in P' \wedge \exists \delta, w.\ \eta = \delta\alpha \wedge w \in T'^{*} \wedge S' \underset{\mathbf{rm}}{\Longrightarrow}_{P'}^{*} \delta A w. \tag{6}$$

Thus validity is just the item-indexed form of viability:

$$\mathsf{Valid}(\eta) \neq \emptyset \iff \eta \text{ is a viable prefix of } G'. \tag{7}$$

Lemma 9 (One-sided LR(0) path invariants)**.** *For arbitrary grammars, the characteristic automaton satisfies the directions needed for LR(0) completeness:*

$$\eta : q_0 \to q \Longrightarrow \mathsf{Valid}(\eta) \subseteq q, \tag{8}$$

$$\mathsf{Valid}(\eta) \neq \emptyset \Longrightarrow \exists q \in Q.\ \eta : q_0 \to q. \tag{9}$$

Moreover, every handle occurrence in a rightmost derivation is realized by a path and by the corresponding completed LR(0) item:

$$\begin{aligned} S' \underset{\mathbf{rm}}{\overset{*}{\Longrightarrow}}_{P'} \eta' A y \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \eta'\omega y, \qquad [A \to \omega] \in P', \qquad y = tz \in T'^* \\ \Longrightarrow \exists p, q \in Q.\ \eta' : q_0 \to p \wedge \omega : p \to q \\ \wedge\, [A \to \omega \cdot \varepsilon] \in q \wedge [A \to \omega] \in \mathbf{reduce}(q, t). \end{aligned} \tag{10}$$

Proof. The key point is that no productivity assumption is needed for the inclusion $\mathsf{Valid}(\eta) \subseteq q$. Prove it simultaneously with the following closure-superset statement: if a kernel contains all valid items at η whose left part is nonempty, and contains the seed $[S' \to \varepsilon \cdot S\$]$ when $\eta = \varepsilon$, then its closure contains every item in $\mathsf{Valid}(\eta)$.

For a valid item with nonempty left part, the preceding dot position is valid at the previous prefix and is put into the predecessor state by induction; the GOTO-kernel then shifts the dot. For a valid item $[A \to \varepsilon \cdot \omega]$, choose the marked occurrence of A in the witness $S' \underset{\mathbf{rm}}{\overset{*}{\Longrightarrow}}_{P'} \eta A w$. If the derivation has length zero, the item is the augmented seed. Otherwise, the step that first introduces this marked occurrence has the form

$$S' \underset{\mathbf{rm}}{\overset{*}{\Longrightarrow}}_{P'} \mu C \rho \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \mu\sigma_1 A \sigma_2 \rho, \qquad [C \to \sigma_1 A \sigma_2] \in P', \qquad \eta = \mu\sigma_1,$$

and the rightmost-occurrence invariant says that the part to the left of the marked A is never changed after that step. The item $[C \to \sigma_1 \cdot A\sigma_2]$ is valid at η by the shorter witness $S' \underset{\mathbf{rm}}{\overset{*}{\Longrightarrow}}_{P'} \mu C \rho$, hence is already in the kernel if $\sigma_1 \neq \varepsilon$, or is obtained by the induction hypothesis if $\sigma_1 = \varepsilon$. One closure step then adds $[A \to \varepsilon \cdot \omega]$. This proves (8) by induction on the path.

For (9), use (7) and induct on $|\eta|$. If $\eta = \varepsilon$, the seed makes $q_0 \neq \emptyset$ and $\varepsilon : q_0 \to q_0$. If $\eta = \eta_0 X$, a valid item at η is either itself a shifted item with left part ending in X, or the closure argument above traces it back to such a shifted valid item. By the induction hypothesis there is $p \in Q$ with $\eta_0 : q_0 \to p$, and (8) puts the predecessor item with dot before X in p. Hence $\mathrm{GOTO}(p, X) \neq \emptyset$, so it lies in Q by Lemma 6, giving the required path.

Finally, the premise of (10) makes $[A \to \omega \cdot \varepsilon]$ valid at $\eta'\omega$. By (9), choose $q \in Q$ with $\eta'\omega : q_0 \to q$, and factor this path uniquely by (4) as $\eta' : q_0 \to p$ and $\omega : p \to q$. Equation (8) puts the completed item in q, and Definition 1(xii) then gives $[A \to \omega] \in \mathbf{reduce}(q, t)$. □

Lemma 10 (LR(0) path exactness under productivity)**.** *Assume every nonterminal of G' is productive. Then every path state is exactly the corresponding valid-item set:*

$$\eta : q_0 \to q \Longrightarrow q = \mathsf{Valid}(\eta). \tag{11}$$

Consequently,

$$(\exists q \in Q)\ \eta : q_0 \to q \Longleftrightarrow \eta \text{ is a viable prefix of } G'. \tag{12}$$

Moreover, every path label is realized as the prefix of a right-sentential form:

$$\eta : q_0 \to q \Longrightarrow \exists z \in T'^*.\ S' \underset{\mathbf{rm}}{\overset{*}{\Longrightarrow}}_{P'} \eta z. \tag{13}$$

Proof. The inclusion $\mathsf{Valid}(\eta) \subseteq q$ is (8). For the reverse inclusion, first prove closure soundness under productivity. Suppose a closure step adds $[A \to \varepsilon \cdot \omega]$ from $[B \to \beta_1 \cdot A\beta_2] \in \mathsf{Valid}(\eta)$, witnessed by $S' \underset{\mathbf{rm}}{\overset{*}{\Longrightarrow}}_{P'} \delta B w$ and $\eta = \delta\beta_1$. Since every nonterminal is productive, the suffix β_2 derives some terminal string $x \in T'^*$; by Lemma 7, this can be done rightmost in the terminal right context. Hence

$$S' \underset{\mathbf{rm}}{\overset{*}{\Longrightarrow}}_{P'} \delta B w \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \delta\beta_1 A \beta_2 w \underset{\mathbf{rm}}{\overset{*}{\Longrightarrow}}_{P'} \delta\beta_1 A x w,$$

so $[A \to \varepsilon \cdot \omega] \in \mathsf{Valid}(\eta)$. Thus the closure of valid kernel items contains no invalid item. The GOTO step preserves validity by shifting the dot:

$$[A \to \alpha \cdot X\beta] \in \mathsf{Valid}(\eta) \Longleftrightarrow [A \to \alpha X \cdot \beta] \in \mathsf{Valid}(\eta X).$$

An induction on $|\eta|$ gives $q \subseteq \mathsf{Valid}(\eta)$ for every $\eta : q_0 \to q$, and therefore (11). Combining (11), (7), and the general viable-implies-path direction (9) proves (12).

For (13), let $\eta : q_0 \to q$. Since $q \in Q$, $q \neq \emptyset$; by (11), choose $[A \to \alpha \cdot \beta] \in \mathsf{Valid}(\eta)$, witnessed by $\eta = \delta\alpha$, $S' \underset{\mathbf{rm}}{\overset{*}{\Longrightarrow}}_{P'} \delta A w$, and $w \in T'^*$. Productivity gives $\beta \Rightarrow^*_{P'} x \in T'^*$, and Lemma 7 rightmostizes this derivation in the terminal right context:

$$S' \underset{\mathbf{rm}}{\overset{*}{\Longrightarrow}}_{P'} \delta A w \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \delta\alpha\beta w \underset{\mathbf{rm}}{\overset{*}{\Longrightarrow}}_{P'} \delta\alpha x w = \eta x w.$$

Thus $z = xw$ witnesses (13). □

Lemma 11 (LR(0) completeness)**.**

$$L(G) \subseteq L(\mathrm{LRA}(G)).$$

Proof. Let $z \in L(G)$, so $S \Rightarrow^*_P z$. Choose a rightmost derivation in G':

$$S\$ = \gamma_0 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_1 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \cdots \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_N = z\$. \tag{14}$$

Write its k-th step as

$$\gamma_k = \alpha_k A_k y_k \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha_k \omega_k y_k = \gamma_{k+1}, \qquad [A_k \to \omega_k] \in P', \quad y_k = t_k z_k \in T'^*. \tag{15}$$

By (10), for each k there exist $p_k, q_k \in Q$ such that

$$\alpha_k : q_0 \to p_k, \qquad \omega_k : p_k \to q_k, \qquad [A_k \to \omega_k \cdot \varepsilon] \in q_k, \qquad [A_k \to \omega_k] \in \mathbf{reduce}(q_k, t_k). \tag{16}$$

Reading (14) from right to left turns each step into one inverse reduction. By (16) and (4), $\alpha_k\omega_k : q_0 \to q_k$, so the Reduce$(A_k \to \omega_k)$ rule applies:

$$(\alpha_k\omega_k : q_0 \to q_k, y_k) \vdash (\alpha_k A_k : q_0 \to \delta(p_k, A_k), y_k) \qquad (k = N-1, \ldots, 0). \tag{17}$$

These reductions are interleaved with Shift blocks. Since $y_k \in T'^*$, the symbol A_k exposed at step k is the rightmost nonterminal of $\gamma_k = \alpha_k A_k y_k$; using the previous step $\gamma_k = \alpha_{k-1}\omega_{k-1}y_{k-1}$ with $y_{k-1} \in T'^*$ and cancelling the common terminal suffix,

$$\alpha_k A_k\, u_k = \alpha_{k-1}\omega_{k-1}, \qquad y_k = u_k\, y_{k-1}, \qquad u_k \in T'^* \qquad (1 \le k \le N-1). \tag{18}$$

By (4) the path $\alpha_{k-1}\omega_{k-1} : q_0 \to q_{k-1}$ factors through $\delta(p_k, A_k)$ after the prefix $\alpha_k A_k$, so the $|u_k|$ symbols of u_k are shifted one by one:

$$(\alpha_k A_k : q_0 \to \delta(p_k, A_k),\ u_k y_{k-1}) \ \vdash^{|u_k|} \ (\alpha_{k-1}\omega_{k-1} : q_0 \to q_{k-1},\ y_{k-1}).$$

At the bottom ($k = N-1$), the run starts by shifting all of $\alpha_{N-1}\omega_{N-1}$ out of $z\$ = \gamma_N$:

$$(\varepsilon : q_0 \to q_0, z\$) \ \vdash^{|\alpha_{N-1}\omega_{N-1}|} \ (\alpha_{N-1}\omega_{N-1} : q_0 \to q_{N-1}, y_{N-1}).$$

At the top ($k = 0$) we have $\gamma_0 = S\$$, whence $\alpha_0 = \varepsilon$, $A_0 = S$, $y_0 = \$$; (17) leaves the stack S reading $\$$, and one final Shift of $\$$ gives

$$(S : q_0 \to \delta(q_0, S), \$) \ \vdash \ (S\$: q_0 \to q_f, \varepsilon), \qquad q_f = \delta(\delta(q_0, S), \$).$$

Concatenating the Shift blocks with (17) for $k = N-1, \ldots, 0$ gives

$$(\varepsilon : q_0 \to q_0, z\$) \vdash^* (S\$: q_0 \to q_f, \varepsilon),$$

hence $z \in L(\mathrm{LRA}(G))$. □

Corollary 12 (Correctness of $\mathrm{LRA}(G)$)**.**

$$L(G) = L(\mathrm{LRA}(G)), \qquad L(G) \triangleq \{z \in T^* \mid S \Rightarrow^*_P z\}.$$

Proof. Corollary 4 and Lemma 11 give

$$L(\mathrm{LRA}(G)) \subseteq L(G), \qquad L(G) \subseteq L(\mathrm{LRA}(G)).$$

□

2 Read and Follow Sets

Define $\mathbf{LA} : \{(q, [A \to \omega]) \mid q \in Q,\ [A \to \omega \cdot \varepsilon] \in q\} \longrightarrow \mathcal{P}(T')$ by

$$\mathbf{LA}(q, [A \to \omega]) \triangleq \left\{ t \in T' \;\middle|\; S' \underset{\mathbf{rm}}{\Longrightarrow}{}^{*}_{P'} \alpha A t z,\ \alpha\omega : q_0 \to q,\ \alpha \in V'^{*},\ z \in T'^{*} \right\}. \tag{19}$$

The remainder of this section computes $\mathbf{LA}(q, [A \to \omega])$ from the LR(0) automaton, as the least fixed point of a Read/Follow digraph.

Let

$$D \triangleq \{(p, A) \mid p \in Q,\ A \in N',\ \delta(p, A) \neq \emptyset\}.$$

For a relation R, write

$$R\,!\,x \triangleq \{y \mid (x, y) \in R\}.$$

Define $\mathbf{Read}, \mathbf{Follow} \subseteq D \times T'$ as the least relations generated by the rules

$$\frac{\delta(\delta(p, A), t) \neq \emptyset}{t \in \mathbf{Read}\,!\,(p, A)}\ \text{DR} \qquad \frac{\delta(p, A) = r \quad C \Rightarrow^{*}_{P'} \varepsilon \quad (r, C) \in D}{\mathbf{Read}\,!\,(r, C) \subseteq \mathbf{Read}\,!\,(p, A)}\ \text{reads}$$

$$\frac{t \in \mathbf{Read}\,!\,(p, A)}{t \in \mathbf{Follow}\,!\,(p, A)}\ \text{Read} \qquad \frac{[B \to \beta \cdot A\gamma] \in p \quad \beta : p' \to p \quad \gamma \Rightarrow^{*}_{P'} \varepsilon \quad (p', B) \in D}{\mathbf{Follow}\,!\,(p', B) \subseteq \mathbf{Follow}\,!\,(p, A)}\ \text{includes}$$

Define the semantic read and follow sets on D:

$$\mathsf{Read}(p, A) \triangleq \{t \in T' \mid \exists r, s, \gamma.\ r = \delta(p, A) \wedge \gamma \Rightarrow^{*}_{P'} \varepsilon \wedge \gamma t : r \to s\}, \tag{20}$$

$$\mathsf{Follow}(p, A) \triangleq \left\{ t \in T' \;\middle|\; \exists \alpha, z.\ S' \underset{\mathbf{rm}}{\Longrightarrow}{}^{*}_{P'} \alpha A t z \wedge \alpha : q_0 \to p \wedge z \in T'^{*} \right\}. \tag{21}$$

For $(p, A) \in D$, set

$$\mathrm{DR}(p, A) \triangleq \{t \in T' \mid \delta(\delta(p, A), t) \neq \emptyset\}.$$

Let

$$\Phi_R(F)(p, A) = \mathrm{DR}(p, A) \cup \bigcup_{\substack{\delta(p,A)=r \\ C \Rightarrow^{*}_{P'} \varepsilon \\ (r,C) \in D}} F(r, C).$$

Lemma 13 (Semantic Read fixed point)**.** *For $(p, A) \in D$,*

$$\mathbf{Read}\,!\,(p, A) = \mu\Phi_R(p, A) = \mathsf{Read}(p, A). \tag{22}$$

Proof. The two **Read**-rules say exactly that **Read**! is the least Φ_R-closed family, i.e. $\mathbf{Read}!(p, A) = \mu\Phi_R(p, A)$; since Φ_R is monotone on the finite lattice $D \to \mathcal{P}(T')$,

$$\mu\Phi_R = \bigcup_{m \geq 0} \Phi_R^m(\emptyset).$$

Write $(p, A) \rightsquigarrow_R (r, C)$ for the reads edge $r = \delta(p, A) \wedge C \Rightarrow^{*}_{P'} \varepsilon \wedge (r, C) \in D$. Because $\Phi_R(\emptyset)(p, A) = \mathrm{DR}(p, A)$, an induction on m unfolds the operator into reads-paths:

$$\Phi_R^{m+1}(\emptyset)(p, A) = \bigcup_{i=0}^{m} \bigcup_{(p,A) \rightsquigarrow_R^{i} (r,C)} \mathrm{DR}(r, C). \tag{23}$$

By (20), $t \in \mathsf{Read}(p, A)$ iff for some nullable $\gamma = C_1 \cdots C_n \in N'^{*}$,

$$r_0 = \delta(p, A), \qquad r_i = \delta(r_{i-1}, C_i)\ (1 \leq i \leq n), \qquad \delta(r_n, t) \neq \emptyset,$$

If $n = 0$, this says exactly that $t \in \mathrm{DR}(p, A)$. If $n > 0$, it is an n-edge reads-path ending at a DR-seed holding t:

$$(p, A) \rightsquigarrow_R (r_0, C_1) \rightsquigarrow_R \cdots \rightsquigarrow_R (r_{n-1}, C_n), \qquad t \in \mathrm{DR}(r_{n-1}, C_n) \quad (\delta(\delta(r_{n-1}, C_n), t) = \delta(r_n, t) \neq \emptyset).$$

In both cases (23) puts t in $\mu\Phi_R(p, A)$, and every member of the union (23) reads back as such a nullable δ-chain. Hence (22). □

Let

$$\Phi_F(F)(p, A) = \mathsf{Read}(p, A) \cup \bigcup_{\substack{[B \to \beta \cdot A\gamma] \in p \\ \beta : p' \to p \\ \gamma \Rightarrow^*_{P'} \varepsilon \\ (p', B) \in D}} F(p', B).$$

Lemma 14 (General Follow over-approximation)**.** *For arbitrary grammars and every $(p, A) \in D$,*

$$\mathsf{Follow}(p, A) \subseteq \mu\Phi_F(p, A) = \mathbf{Follow}\,!\,(p, A). \tag{24}$$

Proof. By Lemma 13, the two **Follow**-rules define the least fixed point of Φ_F. We prove $\mathsf{Follow}(p, A) \subseteq \mu\Phi_F(p, A)$ by induction on a marked rightmost derivation witnessing

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z, \qquad \alpha : q_0 \to p.$$

The occurrence of A cannot be the initial S' occurrence, because it is followed by the terminal t. Let the step that first introduces this marked occurrence be

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' B t' z' \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha' \beta A \gamma t' z' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z, \qquad \alpha = \alpha'\beta, \qquad [B \to \beta A \gamma] \in P'.$$

The marked-occurrence invariant for rightmost derivations says that, after this introducing step, the left context $\alpha'\beta$ and the marked A are never changed. Hence all remaining rewrites occur in the suffix descended from $\gamma t' z'$. Also $[B \to \beta \cdot A\gamma] \in \mathsf{Valid}(\alpha)$; since $\alpha : q_0 \to p$, the one-sided invariant (8) gives

$$[B \to \beta \cdot A\gamma] \in p. \tag{25}$$

Now split by the image of this particular γ-block in the displayed suffix derivation, not by the mere nullability of γ.

If the image of γ is nonempty, let t be its first terminal in the final suffix. Then $\gamma = \gamma_0 X \gamma_1$ where $\gamma_0 \Rightarrow^*_{P'} \varepsilon$ and $X \Rightarrow^*_{P'} t\chi$. Starting from (25), the state $\delta(p, A)$ contains the item $[B \to \beta A \cdot \gamma]$; following the nullable prefix γ_0 and using the FIRST/path extraction lemma gives a path $\gamma_0 t : \delta(p, A) \to s$ for some s. Thus $t \in \mathsf{Read}(p, A)$, and Lemma 13 puts t in the Read seed of $\Phi_F(p, A)$, hence in $\mu\Phi_F(p, A)$.

If the image of γ is empty, then $\gamma \Rightarrow^*_{P'} \varepsilon$ and the first terminal following the marked A is the old t', so $t = t'$. Factor the path $\alpha'\beta : q_0 \to p$ uniquely as $\alpha' : q_0 \to p'$ and $\beta : p' \to p$. The witness $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' B t z$ is strictly shorter than the original marked witness, so the induction hypothesis gives $t \in \mu\Phi_F(p', B)$.

It remains to check that the includes edge is genuinely present. We already have (25), $\beta : p' \to p$, and $\gamma \Rightarrow^*_{P'} \varepsilon$. The fourth premise $(p', B) \in D$ follows from the same marked-occurrence argument. If $B = S'$, then the only S'-production is $S' \to S\$$, and the suffix γ in an item $S' \to \beta A \gamma$ with the dot before A contains \$, hence is not nullable. Thus $B \neq S'$. The live occurrence of B in $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' B t z$ was therefore introduced by some production with the dot before B; that item is valid at α', and (8) puts it in p'. Hence $\delta(p', B) \neq \emptyset$, i.e. $(p', B) \in D$. The includes edge propagates t from (p', B) to (p, A), so $t \in \mu\Phi_F(p, A)$. □

Lemma 15 (Semantic Follow fixed point under productivity)**.** *Assume every nonterminal of G' is productive. For $(p, A) \in D$,*

$$\mathbf{Follow}\,!\,(p, A) = \mu\Phi_F(p, A) = \mathsf{Follow}(p, A). \tag{26}$$

Proof. The inclusion $\mathsf{Follow}(p, A) \subseteq \mu\Phi_F(p, A)$ is Lemma 14. It remains to show that Follow is closed under Φ_F, for then the least fixed point is contained in Follow.

For a Read-seed, fix $t \in \mathsf{Read}(p, A)$. Since $(p, A) \in D$, the state p is reachable from q_0; choose α with $\alpha : q_0 \to p$. By (20), for some nullable γ there is a path $\gamma t : \delta(p, A) \to s$, hence $\alpha A\gamma t : q_0 \to s$. Productivity and Lemma 10 realize this path as a prefix of a right-sentential form: there exists $z \in T'^*$ such that

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A\gamma tz.$$

By Lemma 7, the nullable γ may be erased by rightmost steps in the terminal right context tz, giving

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A\gamma tz \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha Atz.$$

Thus $t \in \mathsf{Follow}(p, A)$.

For an includes edge, assume

$$[B \to \beta \cdot A\gamma] \in p, \qquad \beta : p' \to p, \qquad \gamma \Rightarrow^*_{P'} \varepsilon, \qquad (p', B) \in D,$$

and take $t \in \mathsf{Follow}(p', B)$, witnessed by $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' Btz$ and $\alpha' : q_0 \to p'$. The item supplies the production $B \to \beta A\gamma$, so

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' Btz \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha'\beta A\gamma tz \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha'\beta Atz,$$

where the last segment erases γ in the terminal right context. By (4), $\alpha'\beta : q_0 \to p$, hence $t \in \mathsf{Follow}(p, A)$. Therefore Follow is Φ_F-closed, and the least fixed point equality follows. □

Remark (Marked occurrences for a Coq formalization)**.** The informal phrase "the occurrence introduced by this production" is used in three places: the closure-superset direction of Lemma 9, the converse/over-approximation direction of Lemma 14, and the proof that the includes edge really has $(p', B) \in D$. In a formal development this should be represented once, either as a marked derivation relation or as a zipper over sentential forms.

The reusable invariant is simple: in a rightmost derivation, once a marked occurrence survives a step, the string strictly to its left is never changed by later steps. With this invariant, the introducing step for a marked A has a well-scoped factorization

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' Bt'z' \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha'\beta A\gamma t'z' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha'\beta Atz,$$

and the two cases in Lemma 14 are determined by the actual image of the marked γ-block in this derivation segment. If that image is nonempty, the first terminal comes from a FIRST/path lemma and gives a Read seed. If that image is empty, then $\gamma \Rightarrow^* \varepsilon$, the terminal lookahead is inherited from the shorter B-witness, and the includes edge is available; the separate $(p', B) \in D$ premise follows by applying the same marked-occurrence invariant to the live occurrence of B. Thus the case split is not "γ nullable or not", but "the tracked γ-occurrence has empty image or nonempty image in this particular derivation".

Definition 16 (Digraph lookahead)**.** For every completed item $[A \to \omega \cdot \varepsilon] \in q$, define the lookahead set computed from the Read/Follow graph by

$$\widehat{\mathbf{LA}}(q, [A \to \omega]) \triangleq \{t \in T' \mid \exists p \in Q.\ \omega : p \to q \wedge \delta(p, A) \neq \emptyset \wedge t \in \mathbf{Follow}\,!\,(p, A)\}. \tag{27}$$

Corollary 17 (Digraph lookahead is a safe over-approximation)**.** *For arbitrary grammars, whenever $q \in Q$ and $[A \to \omega \cdot \varepsilon] \in q$,*

$$\mathbf{LA}(q, [A \to \omega]) \subseteq \widehat{\mathbf{LA}}(q, [A \to \omega]). \tag{28}$$

Proof. Take $t \in \mathbf{LA}(q, [A \to \omega])$, witnessed by

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha Atz, \qquad \alpha\omega : q_0 \to q.$$

By (4), factor the path as $\alpha : q_0 \to p$ and $\omega : p \to q$. Then $t \in \mathsf{Follow}(p, A)$ by (21). If $A = S'$, this situation cannot occur because S' occurs on no right-hand side and the initial occurrence is not followed by a terminal in any form derived from S'. Hence $A \neq S'$. Applying the marked-occurrence invariant to the live occurrence of A in $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z$ yields a valid item at α with the dot before A; by (8) that item lies in p. Thus $\delta(p, A) \neq \emptyset$, so $(p, A) \in D$. Finally, Lemma 14 gives $t \in \mathbf{Follow}\,!\,(p, A)$, and the displayed definition of $\widehat{\mathbf{LA}}$ applies. □

Corollary 18 (Lookahead by Follow under productivity)**.** *Assume every nonterminal of G' is productive. Whenever $q \in Q$ and $[A \to \omega \cdot \varepsilon] \in q$,*

$$\mathbf{LA}(q, [A \to \omega]) = \widehat{\mathbf{LA}}(q, [A \to \omega]). \tag{29}$$

Proof. The inclusion $\mathbf{LA} \subseteq \widehat{\mathbf{LA}}$ is Corollary 17. Conversely, suppose $t \in \widehat{\mathbf{LA}}(q, [A \to \omega])$, witnessed by p with $\omega : p \to q$, $\delta(p, A) \neq \emptyset$, and $t \in \mathbf{Follow}\,!\,(p, A)$. By Lemma 15, $t \in \mathsf{Follow}(p, A)$; choose witnesses $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z$ and $\alpha : q_0 \to p$. Then (4) gives $\alpha\omega : q_0 \to q$, and the semantic definition (19) gives $t \in \mathbf{LA}(q, [A \to \omega])$. □

Theorem 19 (Digraph computation of lookahead)**.** *For arbitrary grammars, the Read/Follow digraph computes $\widehat{\mathbf{LA}}$, a safe over-approximation of the semantic lookahead $\mathbf{LA}$. If every nonterminal of G' is productive, then $\widehat{\mathbf{LA}} = \mathbf{LA}$ for every completed item.*

Proof. The Read fixed point is Lemma 13; the general semantic inclusion for Follow is Lemma 14, which gives Corollary 17. Under the explicit productivity hypothesis, Lemma 15 upgrades the inclusion to equality, and Corollary 18 gives the exact semantic formula. □

3 Lookahead Restriction

Recall the semantic lookahead set $\mathbf{LA}$ of (19). The same language-preservation argument below applies to any guard K satisfying

$$\mathbf{LA}(q, [A \to \omega]) \subseteq K(q, [A \to \omega]) \subseteq T'$$

for completed items, because soundness only uses $\mathbf{reduce}_K \subseteq \mathbf{reduce}$ and completeness only uses $\mathbf{LA} \subseteq K$. In particular, for arbitrary grammars one may take the digraph-computed over-approximation $K = \widehat{\mathbf{LA}}$ from Corollary 17; under productivity this is the exact semantic lookahead by Corollary 18. We state the formulas with the exact semantic guard $\mathbf{LA}$. Replace the LR(0) reduction function by

$$\mathbf{reduce}_{\mathbf{LA}}(q, t) \triangleq \{[A \to \omega] \mid [A \to \omega \cdot \varepsilon] \in q,\ t \in \mathbf{LA}(q, [A \to \omega])\}. \tag{30}$$

Let $\vdash_{\mathbf{LA}}$ be the transition relation obtained from $\vdash$ by replacing $\mathbf{reduce}$ with $\mathbf{reduce}_{\mathbf{LA}}$, and let $L(\vdash_{\mathbf{LA}})$ be its accepted language.

Lemma 20 (Guarded soundness)**.**

$$L(\vdash_{\mathbf{LA}}) \subseteq L(G).$$

Proof. From (30),

$$\mathbf{reduce}_{\mathbf{LA}}(q, t) \subseteq \mathbf{reduce}(q, t) \quad \Longrightarrow \quad \vdash_{\mathbf{LA}} \subseteq \vdash \quad \Longrightarrow \quad \vdash_{\mathbf{LA}}{}^* \subseteq \vdash^*.$$

Therefore

$$L(\vdash_{\mathbf{LA}}) \subseteq L(\mathrm{LRA}(G)) = L(G) \qquad \text{by Corollary 12.} \tag{31}$$

□

Lemma 21 (Guarded completeness)**.**

$$L(G) \subseteq L(\vdash_{\mathbf{LA}}).$$

Proof. Let $z \in L(G)$. In the accepting computation constructed in Lemma 11, every Reduce step has the form

$$(\alpha\omega : q_0 \to q, tz') \vdash (\alpha A : q_0 \to \delta(p, A), tz'),$$

where

$$S' \underset{\mathbf{rm}}{\Longrightarrow}_{P'} S\$ \underset{\mathbf{rm}}{\Longrightarrow}^{*}_{P'} \alpha A t z', \qquad \alpha\omega : q_0 \to q, \qquad [A \to \omega \cdot \varepsilon] \in q.$$

By (19),

$$t \in \mathbf{LA}(q, [A \to \omega]) \iff [A \to \omega] \in \mathbf{reduce}_{\mathbf{LA}}(q, t).$$

Thus every Reduce step used in that accepting LR(0) computation is also legal for $\vdash_{\mathbf{LA}}$; Shift steps are unchanged. Hence

$$L(G) \subseteq L(\vdash_{\mathbf{LA}}).$$

□

Corollary 22 (Language preservation)**.**

$$L(\vdash_{\mathbf{LA}}) = L(G).$$

Proof. Combine Lemmas 20 and 21. □

Definition 23 (LALR(1) action table)**.** For a chosen one-symbol guard K on completed items, the ACTION entries are

$$\mathsf{Act}_K(q, t) \triangleq \{\mathsf{shift}(\delta(q, t)) \mid \delta(q, t) \in Q\} \cup \{\mathsf{reduce}(A \to \omega) \mid [A \to \omega \cdot \varepsilon] \in q \wedge t \in K(q, [A \to \omega])\}.$$

A guarded LR(0) table is an LALR(1) table when

$$\forall q \in Q,\ t \in T'. \quad |\mathsf{Act}_K(q, t)| \leq 1.$$

The associated parser is the transition system obtained by executing this single ACTION entry when it exists; if no ACTION entry exists and the configuration is not accepting, the configuration is an error configuration.

Lemma 24 (Conflict-free table condition)**.** *If the resulting table has no shift/reduce or reduce/reduce conflict, its action is single-valued under one-symbol lookahead, hence it is an* LALR(1) *table.*

Proof. At (q, t), the possible actions are

$$\delta(q, t) \quad \text{when this shift target is nonempty,} \qquad \mathbf{reduce}_{\mathbf{LA}}(q, t).$$

Absence of shift/reduce and reduce/reduce conflicts means that this action is single-valued under one-symbol lookahead, which is precisely Definition 23. □

Theorem 25 (Lookahead-guarded reductions)**.** *If the guarded table is conflict-free, it is an* LALR(1) *parser accepting* $L(G)$*.*

Proof. Corollary 22 gives acceptance of $L(G)$, and Lemma 24 gives the LALR(1) table condition under the stated absence of conflicts. □

4 Language-Class Bridge, Corrected

The productivity hypothesis in Theorem 19 is a hypothesis on one concrete grammar and on the LR(0) automaton built from that grammar. It is not a WLOG step inside an automaton-level proof. A productive pruning of a grammar preserves the generated terminal language, but it can change the LR(0) state space by merging states that were separated only by non-productive items.

Consequently, a semantic LALR(1) witness for the original grammar does not, by itself, transport to the directly pruned grammar.

This section states the corrected bridge. There are two separate facts. First, productive pruning preserves the language and yields a grammar to which the exactness theorem $\widehat{\mathbf{LA}} = \mathbf{LA}$ applies. Second, semantic LALR(1) conflict-freeness transports across the pruning only under an additional merge-safety condition, or by running and certifying the pruned parser generator directly. No unconditional nonempty language-class equality is claimed from direct pruning alone.

Definition 26 (Semantic and certified witnesses)**.** A grammar G is a *semantic* LALR(1) *witness* when the ACTION table of Definition 23, built with the exact semantic guard $K = \mathbf{LA}_G$, is conflict-free. A grammar H is a *certified* LALR(1) *witness* when every nonterminal of H' is productive and the ACTION table built with the implemented guard $K = \widehat{\mathbf{LA}}_H$ is conflict-free. Define

$$\mathcal{L}_{\mathrm{sem}} \triangleq \{L(G) \mid G \text{ is a semantic LALR(1) witness}\}, \qquad \mathcal{L}_{\mathrm{cert}} \triangleq \{L(H) \mid H \text{ is a certified LALR(1) witness}\}.$$

The inclusion $\mathcal{L}_{\mathrm{cert}} \subseteq \mathcal{L}_{\mathrm{sem}}$ is immediate from Theorem 19. The reverse inclusion is not proved by direct productive pruning alone.

Definition 27 (Productive pruning)**.** Let $G = (N, T, P, S)$ be an unaugmented grammar. A nonterminal $A \in N$ is *live* when it is productive in the augmented grammar G', ignoring only the fresh start symbol. Thus A is live when

$$A \Rightarrow_P^* w \quad \text{for some } w \in T^*.$$

Assume that S is live. The *productive pruning* G° is the grammar whose nonterminal set is the subtype

$$N^\circ \triangleq \{A \in N \mid A \text{ is live}\},$$

whose terminal set is T, whose start symbol is the live copy of S, and whose productions are exactly the productions

$$A \to X_1 \cdots X_m \in P$$

for which A is live and every nonterminal among the X_i is live, with all live nonterminals retyped into N°. Terminals are left unchanged. The erasure map

$$\pi : N^\circ \uplus T \to N \uplus T$$

forgets the subtype proof on nonterminals and is the identity on terminals.

Lemma 28 (Productive pruning preserves the terminal language)**.** *If the start symbol of G is live, then*

$$L(G^\circ) = L(G).$$

Moreover every nonterminal of $(G^\circ)'$ is productive.

Proof. The inclusion $L(G^\circ) \subseteq L(G)$ follows by erasing the subtype annotations from every production and every derivation step.

For the converse, let $S \Rightarrow_G^* x$ with $x \in T^*$. Every nonterminal occurrence that appears in a derivation which eventually reaches the terminal string x is productive: the suffix of the derivation below that occurrence gives a terminal yield for that nonterminal. Hence every production used along the derivation has a live left-hand side and only live nonterminal occurrences on its right-hand side. The same derivation can therefore be retyped as a derivation in G°, giving $x \in L(G^\circ)$. Equivalently, this is Fact 5 applied only at the language level, not as an automaton-level WLOG step.

Every nonterminal of G° is productive by definition of the subtype N°. Since the start symbol is live, the augmented start symbol of $(G^\circ)'$ is also productive via the fresh production $S'_{G^\circ} \to S^\circ\$$. Thus every nonterminal of $(G^\circ)'$ is productive. □

Definition 29 (Path-indexed image relation)**.** Let $Q^\circ, \delta^\circ, q_0^\circ$ be the LR(0) automaton of $(G^\circ)'$, and let Q, δ, q_0 be the LR(0) automaton of G'. Extend the erasure π to augmented symbols by sending the fresh start and endmarker of $(G^\circ)'$ to the corresponding fresh start and endmarker of G'. For states $q \in Q^\circ$ and $r \in Q$, write

$$q \rightsquigarrow r$$

when there exists a word $\alpha \in {V'_{G^\circ}}^*$ such that

$$\alpha : q_0^\circ \to q \quad \text{in } G^\circ, \qquad \pi(\alpha) : q_0 \to r \quad \text{in } G.$$

This relation is deliberately not a function from pruned states to original states.

Lemma 30 (Path-indexed images step forward)**.** *Assume $q \rightsquigarrow r$ and $\delta^\circ(q, X) = q'$. Then there exists $r' \in Q$ such that*

$$\delta(r, \pi(X)) = r' \qquad \textit{and} \qquad q' \rightsquigarrow r'.$$

Proof. Choose a witnessing path $\alpha : q_0^\circ \to q$ and its erased original path $\pi(\alpha) : q_0 \to r$. Appending the pruned transition by X gives a path $\alpha X : q_0^\circ \to q'$. Erasing the transition gives the original transition by $\pi(X)$, so the erased path ends in $r' \triangleq \delta(r, \pi(X))$. Hence $q' \rightsquigarrow r'$. □

Lemma 31 (There need not be a transition-stable state image)**.** *In general there is no function*

$$\iota : Q^\circ \to Q$$

satisfying

$$\delta^\circ(p, X) = q \quad \Longrightarrow \quad \delta(\iota(p), \pi(X)) = \iota(q)$$

for direct productive pruning.

Proof. Consider the grammar

$$\begin{array}{lcl} S & \to & aA \mid bA \mid aX, \\ A & \to & c, \\ X & \to & cY, \\ Y & \to & Yd. \end{array}$$

The live nonterminals are S and A. Productive pruning removes X and Y, leaving

$$S \to aA \mid bA, \qquad A \to c.$$

In the pruned LR(0) automaton the paths ac and bc reach the same reduce state $q = \{[A \to c \cdot \varepsilon]\}$. In the original automaton, however, the path ac reaches a state containing both

$$[A \to c \cdot \varepsilon] \qquad \text{and} \qquad [X \to c \cdot Y],$$

whereas the path bc reaches a state containing $[A \to c \cdot \varepsilon]$ but not $[X \to c \cdot Y]$. These original target states are different. Thus a single value $\iota(q)$ cannot be compatible with both erased paths, and the transition-stability equation above cannot hold in general. □

Lemma 32 (Direct pruning need not preserve semantic LALR(1) witnesses)**.** *There are grammars G whose semantic* LALR(1) *table is conflict-free, but whose direct productive pruning G° has a semantic* LALR(1) *reduce/reduce conflict.*

Proof. Use the grammar

$$\begin{array}{l} S \to aAd \mid bAe \mid aBe \mid bBd \mid aX \mid bY, \\ A \to c, \\ B \to c, \\ X \to cU, \\ Y \to cV, \\ U \to Uu, \\ V \to Vv. \end{array}$$

The nonterminals X, Y, U, V are non-productive, while S, A, B are productive. In the original LR(0) automaton the dead alternatives split the two relevant states. After reading ac, the state contains the completed items $[A \to c \cdot \varepsilon]$ and $[B \to c \cdot \varepsilon]$, together with the dead item $[X \to c \cdot U]$. The exact semantic lookaheads are

$$\mathbf{LA}_G(A \to c) = \{d\}, \qquad \mathbf{LA}_G(B \to c) = \{e\}$$

in that state. After reading bc, the state is separated by the dead item $[Y \to c \cdot V]$, and the semantic lookaheads are

$$\mathbf{LA}_G(A \to c) = \{e\}, \qquad \mathbf{LA}_G(B \to c) = \{d\}.$$

Thus no reduce/reduce conflict occurs in either original state.

Direct productive pruning removes X, Y, U, V and leaves

$$S \to aAd \mid bAe \mid aBe \mid bBd, \qquad A \to c, \qquad B \to c.$$

Now the two LR(0) states after ac and bc merge into the single core

$$\{[A \to c \cdot \varepsilon], [B \to c \cdot \varepsilon]\}.$$

The semantic lookahead attached to this merged state is the union of the two contexts:

$$\mathbf{LA}_{G^\circ}(A \to c) = \{d, e\}, \qquad \mathbf{LA}_{G^\circ}(B \to c) = \{d, e\}.$$

Hence both reductions are enabled on d and also on e, giving a semantic LALR(1) reduce/reduce conflict in G°. This shows that original semantic conflict-freeness does not automatically transport through direct productive pruning. □

Definition 33 (Table-level merge safety)**.** Let G° be the productive pruning of G. Let $\mathsf{Conflict}_G(r, t)$ mean that the semantic ACTION table of G has a shift/reduce or reduce/reduce conflict at original state r and lookahead t. Define $\mathsf{Conflict}_{G^\circ}(q, t)$ analogously for the semantic ACTION table of the pruned grammar.

The pruning is *table-merge-safe relative to* G when

$$\forall q, t. \quad \mathsf{Conflict}_{G^\circ}(q, t) \Longrightarrow \exists r.\ q \rightsquigarrow r \ \wedge\ \mathsf{Conflict}_G(r, t).$$

Equivalently, every semantic conflict introduced by the pruned table must be the image of a genuine semantic conflict in some original state reachable by an erased pruned path. This condition is a certificate or hypothesis; Lemma 32 shows that it cannot be derived from productive pruning alone.

Definition 34 (Rank-edge merge safety)**.** Some executable developments also certify termination or reduce-edge ordering by a rank on parser states. Such a certificate needs more than a target-state image: a reduce edge records both the predecessor before reading the handle and the state after the reduction. A pruning is *rank-edge merge-safe* when every pruned reduce edge

$$p \xrightarrow{\omega} q, \qquad p \xrightarrow{A} q'$$

can be retargeted to original states

$$r_p \xrightarrow{\pi(\omega)} r_q, \qquad r_p \xrightarrow{A} r_{q'}$$

with $p \rightsquigarrow r_p$, $q \rightsquigarrow r_q$, and $q' \rightsquigarrow r_{q'}$, and with whatever rank decrease the original certificate requires. This is intentionally stated as a separate certificate condition. It should not be hidden inside a canonical state-image function.

Theorem 35 (Merge-safe semantic witness to productive certified witness)**.** *Let G be a grammar with $L(G) \neq \emptyset$, and let G° be its productive pruning. Assume:*

1. *G is a semantic* LALR(1) *witness, and*

2. *the productive pruning G° is table-merge-safe relative to G.*

Then G° is an equivalent productive certified LALR(1) *witness:*

$$L(G^\circ) = L(G), \qquad \text{every nonterminal of } (G^\circ)' \text{ is productive}, \qquad G^\circ \text{ is certified.}$$

If the implementation also requires a termination or reduce-edge rank certificate, add the rank-edge merge-safety condition of Definition 34 to transport that certificate.

Proof. Language equivalence and productivity of $(G^\circ)'$ are exactly Lemma 28. It remains to prove that the implemented LALR(1) table of G° is conflict-free.

First consider the semantic table of G°. If it had a conflict at (q, t), table-merge-safety would give an original state r with $q \rightsquigarrow r$ such that the semantic table of G has a conflict at (r, t). This contradicts the assumption that G is a semantic LALR(1) witness. Hence the semantic table of G° is conflict-free.

Because every nonterminal of $(G^\circ)'$ is productive, Theorem 19 gives

$$\widehat{\mathbf{LA}}_{G^\circ} = \mathbf{LA}_{G^\circ}.$$

Therefore the implemented digraph-computed table and the semantic table of G° have the same enabled reductions. Since the semantic table is conflict-free, the implemented table is conflict-free as well. Thus G° is a certified LALR(1) witness.

The final sentence follows by applying the additional rank-edge merge-safety certificate to each pruned reduce edge. The predecessor state is carried by the path-indexed image relation, not by a global canonical image function, so the required rank decrease is inherited from the original certificate on the retargeted original edge. □

Corollary 36 (Corrected nonempty bridge)**.** *For nonempty languages, the following statements are valid.*

1. $\mathcal{L}_{\mathrm{cert}} \subseteq \mathcal{L}_{\mathrm{sem}}$.

2. *If $L = L(G)$ for some semantic* LALR(1) *witness G, $L(G) \neq \emptyset$, and the productive pruning of G is table-merge-safe, then $L \in \mathcal{L}_{\mathrm{cert}}$.*

There is no unconditional theorem here saying that every nonempty $L \in \mathcal{L}_{\mathrm{sem}}$ is obtained by direct productive pruning as an element of $\mathcal{L}_{\mathrm{cert}}$.

Proof. For (1), if H is certified, every nonterminal of H' is productive. Theorem 19 gives $\widehat{\mathbf{LA}}_H = \mathbf{LA}_H$, so the certified conflict-free table is also the semantic conflict-free table. Hence H is a semantic witness.

For (2), apply Theorem 35 to the chosen grammar G. The resulting productive pruning G° is certified and satisfies $L(G^\circ) = L(G) = L$. □

Remark (Coq-oriented proof shape)**.** In a formalization, the central invariant should be the path-indexed relation $q \rightsquigarrow r$, not a canonical function from pruned states to original states. Canonical images may be introduced only after a separate uniqueness or merge-safety certificate has been supplied. Reduce-edge transport should also carry the predecessor path image explicitly. Otherwise the proof attempts to show a transition-stable quotient property that is false for direct productive pruning in general.

A Digraph Algorithm and Implementation Specification

This appendix gives an executable specification for computing the sets used in Theorem 19. The construction is the usual digraph view of LALR(1) lookahead computation: direct seed sets are

placed on nodes, dependency edges describe set inclusions, and the least fixed point is obtained by collapsing strongly connected components and propagating set unions along the resulting DAG [1, 2, 3].

A.1 General Digraph Fixed-Point Problem

Let X be a finite set of nodes, let $E \subseteq X \times X$ be a directed edge relation, and let $\mathrm{Seed} : X \to \mathcal{P}(T')$. We use the edge orientation

$$x \rightsquigarrow y \qquad \text{to mean} \qquad F(y) \subseteq F(x).$$

Equivalently, define the monotone operator

$$\Psi(F)(x) \triangleq \mathrm{Seed}(x) \cup \bigcup_{x \rightsquigarrow y} F(y).$$

The desired solution is the least fixed point

$$F = \mu\Psi, \qquad F(x) = \mathrm{Seed}(x) \cup \bigcup_{x \rightsquigarrow y} F(y). \tag{32}$$

Verified algorithm: work-list Kleene iteration. For formal verification, take $\mathrm{DIGRAPH}(X, E, \mathrm{Seed})$ to be the following finite work-list computation.

1. Initialize $F(x) \leftarrow \mathrm{Seed}(x)$ for all $x \in X$, and put all edges $x \rightsquigarrow y$ in a work-list.
2. While the work-list is nonempty, remove an edge $x \rightsquigarrow y$. If $F(y) \not\subseteq F(x)$, replace

   $$F(x) \leftarrow F(x) \cup F(y)$$

   and reinsert every incoming edge $u \rightsquigarrow x$.
3. Return F when the work-list is empty.

This is the implementation target used by the specification. It terminates because every update strictly increases some finite set $F(x) \subseteq T'$, so at most $|X| \cdot |T'|$ element insertions occur. The loop invariant is $\mathrm{Seed}(x) \subseteq F(x) \subseteq \mu\Psi(x)$ for all x. When the work-list is empty, every edge satisfies $F(y) \subseteq F(x)$, hence $\Psi(F) \subseteq F$; together with $\mathrm{Seed} \subseteq F$, this says F is Ψ-closed. Since $\mu\Psi$ is the least Ψ-closed family and the invariant gives $F \subseteq \mu\Psi$, the returned value is exactly $\mu\Psi$.

SCC optimization. The usual Tarjan-SCC/condensation-DAG algorithm computes the same fixed point by collapsing cycles and processing the resulting DAG in reverse topological order. It is a valid optimization once accompanied by a separate refinement proof that its result equals the work-list fixed point above. The mathematical development here relies only on the work-list correctness theorem, not on the internal correctness of a particular SCC implementation.

Complexity. Let $b = \lceil |T'|/w \rceil$, where w is the machine word size used for bitsets. The work-list implementation performs at most $|X| \cdot |T'|$ element insertions and scans incident edges when a set grows. With bitsets this is bounded by the usual

$$O\big((|X| + |E|)\,|T'|\,b\big)$$

worst-case work-list bound; the SCC implementation improves the practical bound to $O((|X|+|E|)b)$ after the SCC refinement proof is supplied.

A.2 Instance for Read

First compute nullable nonterminals by the least fixed point

$$\mathrm{Null}(A) \Longleftrightarrow \exists [A \to X_1 \cdots X_k] \in P' \text{ such that every } X_i \text{ is nullable},$$

where terminals are never nullable and $k = 0$ is allowed. Extend this to strings by

$$\mathrm{Null}(X_1 \cdots X_k) \Longleftrightarrow \bigwedge_{i=1}^{k} \mathrm{Null}(X_i).$$

Lemma 37 (Null correctness). *For every $\gamma \in V'^*$,*

$$\mathrm{Null}(\gamma) \Longleftrightarrow \gamma \Rightarrow_{P'}^{*} \varepsilon.$$

In particular, for $A \in N'$, $\mathrm{Null}(A)$ iff $A \Rightarrow_{P'}^{*} \varepsilon$.

Proof. The forward direction is induction on the stage at which a nonterminal enters the least fixed point, followed by concatenating the derivations of the symbols of γ. The reverse direction is induction on the length of a derivation to ε. A terminal cannot occur in such a derivation; for a first step $A \Rightarrow X_1 \cdots X_k \Rightarrow^* \varepsilon$, terminal concatenation decomposition gives $X_i \Rightarrow^* \varepsilon$ for each i, so the induction hypothesis makes every X_i nullable and the fixed-point rule adds A. The string case follows componentwise. □

For $x = (p, A) \in D$, define the direct-read seed

$$\mathrm{DR}(p, A) \triangleq \{t \in T' \mid \delta(\delta(p, A), t) \neq \emptyset\}.$$

Define $E_R \subseteq D \times D$ by

$$(p, A) \rightsquigarrow_R (r, C) \Longleftrightarrow \delta(p, A) = r \wedge C \in N' \wedge \mathrm{Null}(C) \wedge (r, C) \in D.$$

If $(r, C) \notin D$, then $\mathbf{Read}\,!\,(r, C) = \emptyset$, so no edge is needed. Compute

$$\mathbf{Read}\,!\,(p, A) \triangleq \mathrm{DIGRAPH}(D, E_R, \mathrm{DR})(p, A).$$

A.3 Instance for Follow

The seed for follow propagation is the already computed read set:

$$\mathrm{Seed}_F(p, A) \triangleq \mathbf{Read}\,!\,(p, A).$$

Define $E_I \subseteq D \times D$ by

$$(p, A) \rightsquigarrow_I (p', B) \Longleftrightarrow [B \to \beta \cdot A\gamma] \in p \wedge \beta : p' \to p \wedge \mathrm{Null}(\gamma) \wedge (p', B) \in D.$$

The orientation means

$$\mathbf{Follow}\,!\,(p', B) \subseteq \mathbf{Follow}\,!\,(p, A).$$

Compute

$$\mathbf{Follow}\,!\,(p, A) \triangleq \mathrm{DIGRAPH}(D, E_I, \mathrm{Seed}_F)(p, A).$$

A.4 Lookback Relation and Lookahead Construction

For every completed item $[A \to \omega \cdot \varepsilon] \in q$, define

$$\mathrm{LB}(q, A \to \omega) \triangleq \{(p, A) \in D \mid \omega : p \to q\}.$$

Then the implemented digraph guard is

$$\widehat{\mathbf{LA}}(q, [A \to \omega]) \triangleq \bigcup_{(p,A) \in \mathrm{LB}(q, A \to \omega)} \mathbf{Follow}\,!\,(p, A),$$

which is (27) written as an implementation step. It is always a safe over-approximation of the semantic $\mathbf{LA}$, and it is exact under the productivity hypothesis of Corollary 18.

Implementation note. The query $\omega : p \to q$ can be implemented by following δ-edges from p along ω. The query $\beta : p' \to p$ in E_I can be implemented by caching reverse-path queries keyed by (β, p), or by storing predecessor edges during construction of the LR(0) automaton.

A.5 Parser-Table and Conflict Specification

After a guard K has been chosen—for instance $K = \widehat{\mathbf{LA}}$ or, under productivity, $K = \mathbf{LA}$—construct actions as follows.

1. If $\delta(q, t) \neq \emptyset$ for $t \in T'$, add the shift action
$$q \xrightarrow{t} \delta(q, t).$$
2. For each $[A \to \omega \cdot \varepsilon] \in q$ and each $t \in K(q, [A \to \omega])$, add the reduce action $A \to \omega$ under lookahead t.
3. Report a shift/reduce conflict when both a shift action and a reduce action are present for the same pair (q, t).
4. Report a reduce/reduce conflict when two distinct reduce productions are present for the same pair (q, t).

The accepting condition remains the one in Definition 1: the parser accepts when it reaches
$$(S\$: q_0 \to q_f, \varepsilon).$$
Equivalently, in a conventional ACTION/GOTO table, q_f may be treated as the accepting state after the input marker has been consumed.

A.6 Summary Specification

The complete computation required by Theorem 19 is:

1. Build G', Q, and δ as in Definition 1.
2. Compute nullable nonterminals and nullable strings.
3. Build D, DR, and E_R; compute **Read** by DIGRAPH(D, E_R, DR).
4. Build E_I; compute **Follow** by DIGRAPH$(D, E_I, \mathbf{Read})$.
5. Build LB and compute each $\widehat{\mathbf{LA}}(q, [A \to \omega])$ by unioning the appropriate **Follow**-sets.
6. Construct the guarded reduce actions and check conflicts.

The postcondition is that the computed **Read**, **Follow**, and $\widehat{\mathbf{LA}}$ are the least sets described by Theorem 19. For arbitrary grammars, $\widehat{\mathbf{LA}}$ safely over-approximates the semantic lookahead and therefore preserves the language when used as the guard in Section 3. Under the explicit productivity hypothesis, $\widehat{\mathbf{LA}} = \mathbf{LA}$, so the computed sets are exactly the semantic LALR(1) lookahead sets determined by the LR(0) automaton.

B Fuel-Free Parser Execution

The table specification above is relational. An executable parser can be obtained without adding a fuel argument exactly when no reachable reduce-only phase can run forever. This section states that condition using the actual parser stacks, not a finite control-only over-approximation.

Definition 38 (Run stacks and reachable phase heads)**.** A run stack is a viable stack path starting at the initial LR state:

$$\mathbf{Stack} \triangleq \{\alpha : q_0 \to q \mid \alpha \in V'^*,\ q \in Q,\ \alpha : q_0 \to q\}.$$

The corresponding run configurations are

$$\mathbf{Config}_{\text{run}} \triangleq \{(s, u) \mid s \in \mathbf{Stack},\ u \in T'^*\}.$$

For an input word $w \in T^*$, put

$$c_0(w) \triangleq (\varepsilon : q_0 \to q_0, w\$).$$

A run configuration is reachable when

$$\mathsf{Reach}(c) \iff \exists w \in T^*.\ c_0(w) \vdash^*_{\mathbf{LA}} c.$$

For $s \in \mathbf{Stack}$ and $t \in T'$, say that (s, t) is a reachable reduce-phase head when

$$\mathsf{Phase}(s, t) \iff \exists u \in T'^*.\ \mathsf{Reach}(s, tu).$$

Definition 39 (Exact reduce-phase relation)**.** For each lookahead $t \in T'$, define a relation $\Rightarrow_t \subseteq \mathbf{Stack} \times \mathbf{Stack}$ by

$$s \Rightarrow_t s'$$

iff there exist $\alpha \in V'^*$, $A \in N'$, $\omega \in V'^*$, and states $p, q, q' \in Q$ such that

$$s = (\alpha\omega : q_0 \to q), \qquad s' = (\alpha A : q_0 \to q'),$$

$$\alpha : q_0 \to p, \qquad \omega : p \to q, \qquad [A \to \omega] \in \mathbf{reduce}_{\mathbf{LA}}(q, t), \qquad q' = \delta(p, A).$$

Thus $s \Rightarrow_t s'$ records one actual guarded Reduce step with lookahead t, including the stack prefix that is exposed by popping the handle.

Lemma 40 (Exactness of reduce phases)**.** *For $s, s' \in \mathbf{Stack}$, $t \in T'$, and $u \in T'^*$, the following are equivalent:*

$$(s, tu) \vdash_{\mathbf{LA}} (s', tu) \quad \textit{by a Reduce step}$$

and

$$s \Rightarrow_t s'.$$

Consequently $\Rightarrow_t$ is neither forgetting the pop predecessor nor combining incompatible stack contexts.

Proof. Expanding the guarded Reduce rule, a Reduce step from (s, tu) to (s', tu) has the form

$$s = (\alpha\omega : q_0 \to q), \qquad s' = (\alpha A : q_0 \to q'),$$

with witnesses $p \in Q$, $A \in N'$, and $\omega \in V'^*$ satisfying

$$\alpha : q_0 \to p, \qquad \omega : p \to q, \qquad [A \to \omega] \in \mathbf{reduce}_{\mathbf{LA}}(q, t), \qquad q' = \delta(p, A).$$

These are exactly the witnesses required by Definition 39. The converse is the same argument read backwards. □

Definition 41 (Exact reduce-phase termination)**.** For a relation R, write $\mathrm{Acc}_R(x)$ for accessibility of x: every R-successor of x is again accessible. The guarded table is *phase-terminating* when every reachable reduce-phase head is accessible for the exact reduce relation:

$$\forall s \in \mathbf{Stack},\ t \in T'.\quad \mathsf{Phase}(s, t) \Longrightarrow \mathrm{Acc}_{\Rightarrow_t}(s),$$

where $\mathrm{Acc}_{\Rightarrow_t}(s)$ uses the successor relation $s \Rightarrow_t s'$. This is the constructive well-foundedness formulation. The classically equivalent "there is no infinite reachable $\Rightarrow_t$-chain" formulation is not used as the definition.

Lemma 42 (Finite branching of exact reductions). *For each $s \in \mathbf{Stack}$ and $t \in T'$, the set*

$$\{s' \in \mathbf{Stack} \mid s \Rightarrow_t s'\}$$

is finite. If the guarded table is conflict-free, this set has at most one element.

Proof. Write $s = \beta : q_0 \to q$. A successor of s under $\Rightarrow_t$ must be produced by some completed item $[A \to \omega] \in \mathbf{reduce_{LA}}(q, t)$. This set of items is finite. For a fixed $A \to \omega$, the equality $\beta = \alpha\omega$ determines at most one prefix α, and the deterministic LR path from q_0 along α determines at most one exposed state p. Then the successor state is forced to be $\delta(p, A)$. Hence only finitely many successors can occur. If the table is conflict-free, at most one Reduce action is available at the pair (q, t), so there is at most one successor. □

Lemma 43 (Heights for finitely branching accessible relations). *Let $R \subseteq X \times X$ be finitely branching, with a finite successor list for each x. If $\mathrm{Acc}_R(x)$, then one can define, by well-founded recursion on this accessibility proof,*

$$H_R(x) \triangleq \begin{cases} 0, & \text{if } x \text{ has no } R\text{-successor}, \\ 1 + \max\{H_R(y) \mid x\ R\ y\}, & \text{otherwise.} \end{cases}$$

Moreover, if $x\ R\ y$, then $H_R(y) < H_R(x)$. Conversely, any map $H : X \to \mathbb{N}$ that strictly decreases along R gives $\mathrm{Acc}_R(x)$ for every x.

Proof. Accessibility gives the recursive calls $H_R(y)$ for every successor $x\ R\ y$; finite branching supplies the finite maximum. The definition therefore immediately yields $H_R(x) \geq 1 + H_R(y)$ for each successor, so $H_R(y) < H_R(x)$. Conversely, if H strictly decreases along R, prove $\mathrm{Acc}_R(x)$ by induction on $H(x)$: every successor has strictly smaller rank and is accessible by the induction hypothesis. □

Definition 44 (Exact termination certificate). For a conflict-free guarded table, a termination certificate is a rank function

$$\rho : \mathbf{Config}_{\mathrm{run}} \to \mathbb{N}$$

such that every reachable Shift or Reduce step strictly decreases

$$\mu(c) \triangleq (|\mathrm{input}(c)|, \rho(c)) \in \mathbb{N} \times \mathbb{N}$$

in lexicographic order. Concretely, whenever $\mathsf{Reach}(c)$ and $c \vdash_{\mathbf{LA}} c'$ is a Shift or Reduce step, one has

$$\mu(c') <_{\mathrm{lex}} \mu(c).$$

Accept and Error configurations make no recursive call.

Definition 45 (Recursive-call relation). Let $\mathsf{Call} \subseteq \mathbf{Config}_{\mathrm{run}} \times \mathbf{Config}_{\mathrm{run}}$ be the relation

$$c'\ \mathsf{Call}\ c$$

iff $\mathsf{Reach}(c)$, $c \vdash_{\mathbf{LA}} c'$, and that one parser step is either Shift or Reduce. Accept and Error configurations have no Call successors.

Lemma 46 (Certificate gives accessibility). *If a termination certificate ρ exists, then every reachable run configuration c satisfies*

$$\mathrm{Acc}_{\mathsf{Call}}(c).$$

Equivalently, the recursive parser may make recursive calls only to Call*-smaller configurations.*

Proof. Put $\mu(c) = (|\mathrm{input}(c)|, \rho(c))$. By Definition 44, every Call-edge c' Call c satisfies

$$\mu(c') <_{\mathrm{lex}} \mu(c).$$

The lexicographic order on $\mathbb{N} \times \mathbb{N}$ is well-founded, and well-foundedness is preserved by inverse image along μ. Hence the measure order

$$c' \prec_\rho c \quad \Longleftrightarrow \quad \mu(c') <_{\mathrm{lex}} \mu(c)$$

is well-founded on run configurations. Since Call is a subrelation of $\prec_\rho$ on reachable configurations, every reachable configuration is accessible for Call. □

Definition 47 (Canonical exact rank)**.** Assume the guarded table is phase-terminating. For $s \in \mathbf{Stack}$ and $t \in T'$, define $H_t(s)$ as follows. If $\mathsf{Phase}(s, t)$, use the accessibility proof supplied by Definition 41 and Lemma 43 for $\Rightarrow_t$. If $\neg\mathsf{Phase}(s, t)$, put $H_t(s) = 0$. For a run configuration $c = (s, u)$, put

$$\rho_{\mathrm{ex}}(c) \triangleq \begin{cases} H_t(s), & u = tu' \text{ for some } t \in T', u' \in T'^*, \\ 0, & u = \varepsilon. \end{cases}$$

This rank is a proof object: because Reach and Phase contain existential witnesses, ρ_{ex} is not by itself a complete decision procedure for arbitrary tables.

Theorem 48 (Sound and complete fuel-free criterion)**.** *For a conflict-free guarded table, the following implications hold constructively:*

(a) phase-termination in the sense of Definition 41 implies the existence of a termination certificate;

(b) a termination certificate implies that every public parser run starting from some $c_0(w)$, $w \in T^*$, is finite.

Conversely, in classical mathematics, finite public runs imply phase-termination; hence the three conditions are equivalent classically. When phase-termination holds, the canonical exact rank ρ_{ex} of Definition 47 is a termination certificate.

Proof. Phase-termination implies a certificate. Assume phase-termination. Let $c = (s, u)$ be reachable. If $u = tu'$, then $\mathsf{Phase}(s, t)$. By Definition 41, s is accessible for $\Rightarrow_t$; Lemma 42 and Lemma 43 give a height H_t that strictly decreases along every exact reduce edge.

We prove that ρ_{ex} is a termination certificate. If the recursive step from c to c' is a Shift and $\mathrm{input}(c) = tu'$, then $\mathrm{input}(c') = u'$, so

$$|\mathrm{input}(c')| < |\mathrm{input}(c)|.$$

Therefore $\mu(c') <_{\mathrm{lex}} \mu(c)$, independently of the second component. If the recursive step is a Reduce, write

$$c = (s, tu'), \qquad c' = (s', tu').$$

By Lemma 40, $s \Rightarrow_t s'$. The height decrease gives

$$\rho_{\mathrm{ex}}(c') = H_t(s') < H_t(s) = \rho_{\mathrm{ex}}(c).$$

The input length is unchanged by a Reduce step, so again $\mu(c') <_{\mathrm{lex}} \mu(c)$. Accept and Error configurations make no recursive call. Thus ρ_{ex} is a termination certificate.

A certificate implies finite public runs. Assume a termination certificate ρ exists. Along every reachable recursive parser step, the measure

$$\mu(c) = (|\mathrm{input}(c)|, \rho(c))$$

strictly decreases in the lexicographic order on $\mathbb{N} \times \mathbb{N}$. This order is well-founded, so no infinite sequence of reachable recursive calls can start from any $c_0(w)$. Hence every public parser run is finite.

Classical converse. Assume classical dependent choice, or equivalently the standard finitely branching form of Kőnig's lemma. If phase-termination failed, then some reachable phase head (s_0, t) would be non-accessible; finite branching would produce an infinite chain

$$s_0 \Rightarrow_t s_1 \Rightarrow_t s_2 \Rightarrow_t \cdots .$$

By $\mathsf{Phase}(s_0, t)$, choose u and w with $c_0(w) \vdash^*_{\mathbf{LA}} (s_0, tu)$. Lemma 40 turns each edge of the chain into a guarded Reduce step on the same unread input tu, so appending the infinite reduce phase to the finite prefix gives an infinite public parser run. Therefore, classically, finite public runs imply phase-termination. □

Corollary 49 (Fuel-free recursive recognizer)**.** *If the guarded table has a termination certificate, the public recognizer can be exposed without a fuel parameter:*

$$\text{run\_parser} : T^* \to \text{bool}.$$

It returns true *exactly for accepted inputs. A parser returning* option ParseTree *requires a separate tree-carrying configuration type and a proof that the produced tree denotes the corresponding derivation; that structure is not part of the recognizer defined in this note.*

Proof. Use the certificate rank to order recursive calls by

$$c' \prec c \quad \Longleftrightarrow \quad \mu(c') <_{\text{lex}} \mu(c).$$

The lexicographic order on $\mathbb{N}\times\mathbb{N}$ is well-founded, and the certificate supplies the required decrease for every reachable Shift or Reduce call. Thus the internal runner is defined on reachable configurations by well-founded recursion on $\prec$, equivalently by carrying a reachability proof internally. The initial configuration $c_0(w)$ is reachable by the empty run, so the public wrapper supplies the accessibility proof and exposes no fuel argument. Its result is Boolean because the configurations above carry only control and input, not parse-tree fragments. □

The criterion above is exact as a mathematical specification: it rejects a table only when there is an actual infinite reduce-only phase reachable from some initial input. It is not, by itself, a complete executable decision procedure, because Reach, Phase, and the canonical rank ρ_{ex} contain existential proof data. A finite control-only graph may still be useful as a sufficient quick check, but it is not complete because such a graph can contain cycles that no single parser stack can realise. For an executable Coq development one should either use such sufficient checks or supply a table-specific termination certificate.

References

[1] F. DeRemer and T. J. Pennello. Efficient computation of LALR(1) look-ahead sets. *ACM Transactions on Programming Languages and Systems*, 4(4):615–649, 1982.

[2] R. E. Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.

[3] J. H. Gallier. A survey of LR-parsing methods: The graph method for computing fixed points and LALR(1) lookahead sets. CIS 511 lecture notes, University of Pennsylvania, 2008.